

Server-Factory-Core-Framework

すべての**Server Factory**を支える共通エンジン

概要

Core Frameworkは、Server Factoryファミリーのプロビジョニングツールを支えるKotlinフレームワークです。Mail Server Factoryなどのプロジェクトが基盤とする共通エンジンと抽象化レイヤーを提供し、各「ファクトリー」が個別にプロビジョニングの基本機能を再実装するのではなく、実績のある一つの基盤を再利用できるようにします。

簡単な説明

Server Factoryエコシステムの基盤となる共通のKotlinフレームワーク。下流のファクトリー（Mail Server Factory、Web Service Factory、SonarQube Factoryなど）が利用する共通のプロビジョニングエンジン、接続抽象化レイヤー、インストールステップの機構を提供します。

詳細な説明

Core Frameworkは、Server-Factoryファミリー全体を可能にする縁の下の力持ちです。個々の「ファクトリー」製品（Mail Server Factory、Web Service Factory、SonarQube Factory、Caching Proxy Factory）はすべて、この再利用可能なエンジンの上に構築されています。Server Factoryのアプローチは宣言型です。ユーザーは望むインフラを設定ファイルとして記述し、ファクトリーはその記述を解釈してターゲットシステムにソフトウェアをインストール・初期化します。そして、このパターンに共通する機構が実際に実装されているのがCore Frameworkです。具体的には、あらゆる種類のターゲットに到達するための接続・転送抽象化レイヤー、ソフトウェアがどのようにプロビジョニングされるかを定義するインストールステップモデル、そして各ファクトリーが個別に実装する必要のあった共通の基盤部分がここに集約されています。

これは、複数の製品を持つツールチェーンがいずれ直面する構造的な問い——「共通エンジンはどこに置くべきか」——に対する答えです。この答えを一度正しく出すことで、ファミリーは4つの微妙に異なるプロビジョ

ナーに分裂することなく、一貫性を保っています。このKotlinフレームワークに集約することで、ファミリーはプロビジョニングロジックの重複を避け、動作の一貫性を維持します。Core Frameworkで改善された接続タイプやインストールの基本機能は、すべての下流ファクトリーに恩恵をもたらします。

その実態はほぼKotlin（約990KBのKotlinコードに薄いShellレイヤーを加えたもの）で、スクリプト集ではなくコードライブラリとしての役割を反映しています。下流のリポジトリはすべて、このCore Frameworkを標準的な依存関係として参照しています（Parallels-Utils、Qemu-Utils、Utils、Definitionsパックはすべて、Core Frameworkリポジトリをエコシステムの中心としてリンクしています）。READMEは意図的に最小限に抑えられており——他のプロジェクトのためのインフラであり、`version.txt/version_code.txt`でバージョン管理されています——また、後のAIの取り組みよりも前に開発されたため、組織の成熟したDevOpsツールチェーンの遺産の一部となっています。

なぜ開発したのか

各プロビジョニングツールには、ターゲットへの接続手段とソフトウェアのインストール・設定ステップという共通の核が必要です。これを製品ごとに再構築すれば、動作のばらつきやバグの増加を招きます。Core Frameworkはそれを一箇所に集約し、すべてのファクトリーが信頼できる一つのエンジンを共有できるようにします。

なぜ革新的なのか

これはファミリー全体で最も影響力の大きい部分です。ここで強化された接続タイプや改善されたインストールの基本機能は、すべてのファクトリーに即座に反映され、ツールチェーン全体が一つの投資から複利効果を得ることができます。「一度作って、どこでも再利用する」という哲学が最も効果を発揮する場所——インフラ自動化の基盤レイヤーで、正しい場所での修正が下流のすべてを修正するのです。

コンテンツ

革新的なポイント

- 接続とインストール手順のロジックを抽象化した、再利用可能な単一のプロビジョニング・フレームワーク。
- エンジン（コア・フレームワーク）と製品固有のファクトリー間の明確な分離。
- 再現性のある利用を実現するバージョン固定配布（`version.txt/version_code.txt`）。

課題と解決策

- **プロビジョニング・ロジックの重複回避**: 全ファクトリーで共有されるフレームワークに共通機構を抽出することで解決。
- **製品間での動作の一貫性**: 接続タイプと手順がどこでも同一に動作するよう、共通の抽象化を実施。
- **(未検証)**: 特定の内部APIは公開READMEに記載されていない。インターフェースの詳細は「ファクトリーで共有されるフレームワーク」を超えて未検証とみなすこと。

技術スタック（採用理由と使用方法）

- **Kotlin** — フレームワーク全体（約990KB）。Server Factoryファミリーの言語。
- **Shell** — 最小限の補助スクリプト。
- **Gradle** — ビルドツールチェーン（ファミリーの./gradlew使用と一貫性あり）。

注: GitHubは、Server-Factory組織内のフォークとしてリポジトリを示す。AI中心ではなく、プロビジョニング・ツールチェーンの基盤として提示。